

MANUAL TÉCNICO

Arquitectura e Intérprete del Lenguaje GoLite

Estudiante: Jhostin Javier Ramírez Enríquez

Carnet: 202404407

Curso: Organización de Lenguajes y Compiladores 1

Fecha: Junio 2026

Institución: Universidad de San Carlos de Guatemala

1. Arquitectura del Intérprete

El compilador/intérprete de GoLite se implementó bajo la plataforma de desarrollo Java empleando un pipeline tradicional dividido en fases acopladas para un lenguaje con tipado estricto:

- 1. **Analizador Léxico (JFLEX):** Segmenta la cadena de entrada de texto en tokens reconocibles, descartando los espacios en blanco y los comentarios simples (`//`) o de bloque (`/* ... */`).
- 2. **Analizador Sintáctico (CUP):** Evalúa el orden de los tokens utilizando un algoritmo de análisis ascendente LALR basado en la gramática formal provista, encargándose de modelar la jerarquía del Árbol de Sintaxis Abstracta (AST).
- 3. **Analizador Semántico:** Valida el cumplimiento de restricciones lógicas, control de tipos estáticos compatibles en operaciones e inicialización de entornos/ámbitos.
- 4. **Ejecución (Patrón Interpreter):** Realiza un recorrido ordenado sobre los nodos lógicos del AST a partir de la declaración de `func main()`.

2. Especificación de Tipos de Datos Nativos

GoLite maneja un estricto control de tipos estáticos, lo que impide asignaciones implícitas incompatibles. A continuación se presentan las características operativas de sus tipos:

Tipo Primitivo	Descripción Operativa	Valor Inicial por Defecto
int	Enteros tradicionales de 32 bits	0
float64	Números de coma flotante de doble precisión	0.0
string	Cadenas de texto envueltas en comillas dobles ("")	""
bool	Operadores booleanos de lógica (true / false)	false
rune	Representación de carácter unicode literal ('a')	0

3. Estructuras Avanzadas y Gestión de Memoria

Las colecciones dinámicas (*Slices*) y los tipos definidos por el usuario (*Structs*) se inicializan de forma perezosa en memoria. Si no se les asignan datos literales al momento de su declaración, asumen por defecto la referencia nula universal de lenguaje: `nil`. El paso de instancias de *Structs* o *Slices* hacia funciones se realiza de forma obligatoria por `**referencia**`.

4. Estrategia de Manejo y Recuperación de Errores

El analizador incluye un sistema de gestión de excepciones para registrar fallas concurrentes sin interrumpir el funcionamiento general del entorno. La clase genérica estructural para almacenar cada anomalía es la siguiente:

```
public class TokenError {  
    private String tipoError; // LÉXICO, SINTÁCTICO o SEMÁNTICO  
    private int linea;        // Fila exacta del archivo fuente  
    private int columna;      // Columna exacta del archivo fuente  
    private String mensaje;    // Descripción detallada del problema  
}
```